

피드백

날짜	@2026년 3월 19일
상태	진행 중

- AI Agent 포함 모든 내용에 대해서 → 자세하게 개선 사항 정의
- 이를 AI에게 줘서 그대로 수정하길 바람
- 시험 생성 관련해서 MergeEDUAgent에 대한 사항이 나오는데 → 직접 시험 관련 내용을 써보면서 그대로 적용할 것
 - env파일에 API키 최신화해서 해야함
- Gemini API를 사용할 일이 있는데 다음 docs를 활용할 것
 - https://ai.google.dev/gemini-api/docs/text-generation?hl=ko&gl=1*1btc5pl*_up*MQ...gaMTM0OTQzNTc4Mi4xNzc0MDgyMzQ2*_ga_P1DBVKWT6V*cze3NzQwODIzN
 - 특히 PDF 입력은 pathlib 사용 방식을 써야함

▼ 예시 코드

```
from google import genai
from google.genai import types
import io
import httpx

client = genai.Client()

long_context_pdf_path = "https://www.nasa.gov/wp-content/uploads/static/history/alsj/a17/A17_FlightPlan.pdf"

# Retrieve and upload the PDF using the File API
doc_io = io.BytesIO(httpx.get(long_context_pdf_path).content)

sample_doc = client.files.upload(
    # You can pass a path or a file-like object here
    file=doc_io,
    config=dict(
        mime_type='application/pdf'
    )
)

prompt = "Summarize this document"

response = client.models.generate_content(
    model="gemini-3-flash-preview",
    contents=[sample_doc, prompt])
print(response.text)
```

강의 에이전트

강의 에이전트 개선안

- 현재 강의 에이전트는 솔직히 실사용이 어려운 수준이다
- 따라서 당장은 에이전트 기능을 고도화하는 방향보다, 에이전트를 배제한 단순한 구조로 먼저 적용하는 방향이 더 현실적이다
- 목표는 복잡한 자율 에이전트 구현이 아니라, 사용자가 PDF를 따라가며 자연스럽게 설명을 듣고 상호작용할 수 있는 강의 보조 시스템을 만드는 것이다

핵심 방향

- 기존의 에이전트형 기능은 일단 폐기한다
- 대신, 사용자가 현재 보고 있는 PDF 페이지를 기준으로 설명을 제공하는 구조로 단순화한다
- 설명 흐름은 시스템이 주도하되, 페이지 이동과 추가 질문은 사용자 의사에 따라 진행되도록 한다
- 결과적으로, 복잡한 에이전트 없이도 실제로 강의를 듣는 데에는 문제가 없도록 만드는 것이 목적이다

1. PDF 뷰어 동작 방식 고정 (완료 - 3.19)

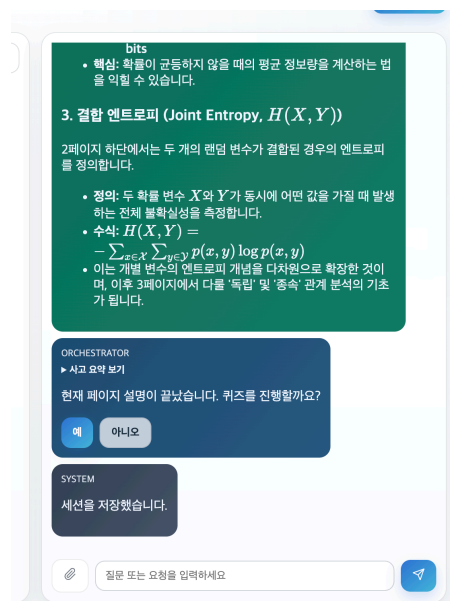
- 우선 PDF 뷰어를 스크롤 형식이 아니라 좌우 슬라이드 형식으로 고정한다.
- 한 화면에는 페이지 하나만 표시한다.
- 현재 어떤 페이지를 보고 있는지는 별도의 변수로 저장한다.
- 이렇게 해야 시스템이 사용자가 현재 보고 있는 페이지를 명확하게 추적할 수 있고, 해당 페이지를 기준으로 설명을 제공하기 쉬워진다.

2. 설명 생성 방식

- Gemini에는 다음 정보를 입력으로 전달한다.
 - PDF 파일
 - 해당 내용을 직관적으로 이해하기 쉽게 설명해 달라는 프롬프트
 - 현재 사용자가 보고 있는 페이지 번호
- 시스템 프롬프트에는 다음 조건을 명시한다.
 - 현재 페이지 위주로 설명해야 한다
- 즉, 전체 PDF를 두루뭉술하게 요약하는 것이 아니라, 사용자가 지금 보고 있는 페이지를 중심으로 설명하도록 한다.

3. 페이지 설명 이후의 사용자 흐름

- 현재 페이지에 대한 설명이 끝나면, 사용자에게 다음 선택지를 제공한다.
 - 다음 페이지로 넘어가기 버튼
 - 또는 아니오



- 버튼의 느낌은 대화창에서 이러한 느낌이 나도록 해야한다.
- 사용자가 아니오를 선택한 경우, 시스템은 사용자가 추가 질문을 하도록 유도해야 한다.
- 전체 흐름은 다음과 같다.

- 현재 페이지 설명 제공
- “다음 페이지로 넘어갈까요?”라는 선택 제공
- 사용자가 아니오를 누르면 추가 질문 유도
- 추가 질문이 끝나면 다시 “다음으로 넘어갈까요?”라고 질문

4. 사용자가 아무 반응이 없을 때의 처리

- 처음에 사용자가 아니오 버튼을 누른 뒤, 추가 질문도 하지 않고 계속 아무 동작도 하지 않는 경우가 있을 수 있다.
- 이 경우 시스템은 한 번 더 다음과 같이 묻는다.
 - “다음 페이지로 넘어갈까요?”
- 즉, 사용자가 흐름에서 멈추지 않도록 한 번 정도는 재진입을 유도하는 장치를 둔다.

5. 다음 페이지 이동 시 동작

- 사용자가 다음 페이지로 넘어가기 버튼을 누르면, 자동으로 다음 페이지에 대한 설명이 시작되어야 한다.
- 이때 다음 두 동작이 함께 일어나야 한다.
 - 대화에서는 다음 페이지 설명이 시작되어야 한다.
 - PDF 뷰어 역시 실제로 다음 페이지 화면으로 이동해야 한다.
- 즉, 대화 상태와 PDF 뷰어 상태가 항상 동기화되어야 한다.

6. 사용자가 직접 페이지를 이동하는 경우

- 사용자는 시스템 버튼이 아니라, PDF 뷰어 자체의 페이지 이동 버튼으로 임의로 이전 또는 다음 페이지로 이동할 수 있다.
- 이 경우의 처리 방식은 다음과 같다.

6-1. 기존 답변이 출력 중이던 경우

- 현재 어떤 페이지에 대한 답변이 출력 진행 중이었다면, 그 답변은 일단 그대로 끝까지 출력한다.
- 그 이후에 새로 이동한 페이지를 기준으로 다음과 같이 묻는다.
 - “해당 페이지에 대해서 설명할까요?”
- 즉, 페이지가 중간에 바뀌더라도 이미 생성 중인 응답을 끊지 않고, 현재 응답을 마무리한 뒤 다음 상호작용으로 연결한다.

6-2. 설명이 진행 중이 아니던 경우

- 별도의 설명이 진행 중이 아니었다면, 사용자가 페이지를 이동한 직후 바로 다음과 같이 묻는다.
 - “현재 페이지에 대해서 설명할까요?”
- 즉, 사용자의 페이지 이동을 새로운 설명 시작 신호로 해석하는 구조다.

6-3. 빠르게 훑어보는 상황 예외 처리

- 다만 사용자가 빠르게 페이지를 훑어보는 중일 수도 있다.
- 예를 들어, 사용자가 페이지 이동 버튼을 누른 뒤 1초 이내에 또 다음 페이지로 이동했다면, 이는 해당 페이지를 자세히 보려는 것이 아니라 빠르게 훑어보는 행동으로 해석할 수 있다.
- 이 경우에는 그 페이지에 대해 다음 UI를 출력하지 않는다.
 - “설명할까요?”
- 즉, 짧게 스쳐 지나간 페이지마다 설명 여부를 묻지 않도록 해서 불필요한 인터럽션을 줄인다.
- 여기서 말하는 페이지 이동 버튼은 PDF 뷰어에 있는 페이지 이동 버튼을 의미한다.

7. 스트리밍 출력

- 문서 확인
 - https://ai.google.dev/gemini-api/docs/text-generation?hl=ko&gl=1*9ifw7c*_up*MQ

```

from google import genai

client = genai.Client()

response = client.models.generate_content_stream(
    model="gemini-3-flash-preview",
    contents=["Explain how AI works"]
)
for chunk in response:
    print(chunk.text, end="")

```

7.1 사고 요약 출력

- 해당 문서의 생각 요약 부분 확인
 - https://ai.google.dev/gemini-api/docs/thinking?hl=ko&gl=1*2bbki7*_up*MQ.._gaMTI2MzQ0MDIyNi4xNzczODkyMjQ0*_ga_P1DBVKWT6V*czE3NzM4OTIyNDG

7.2 스트리밍 출력 시나리오

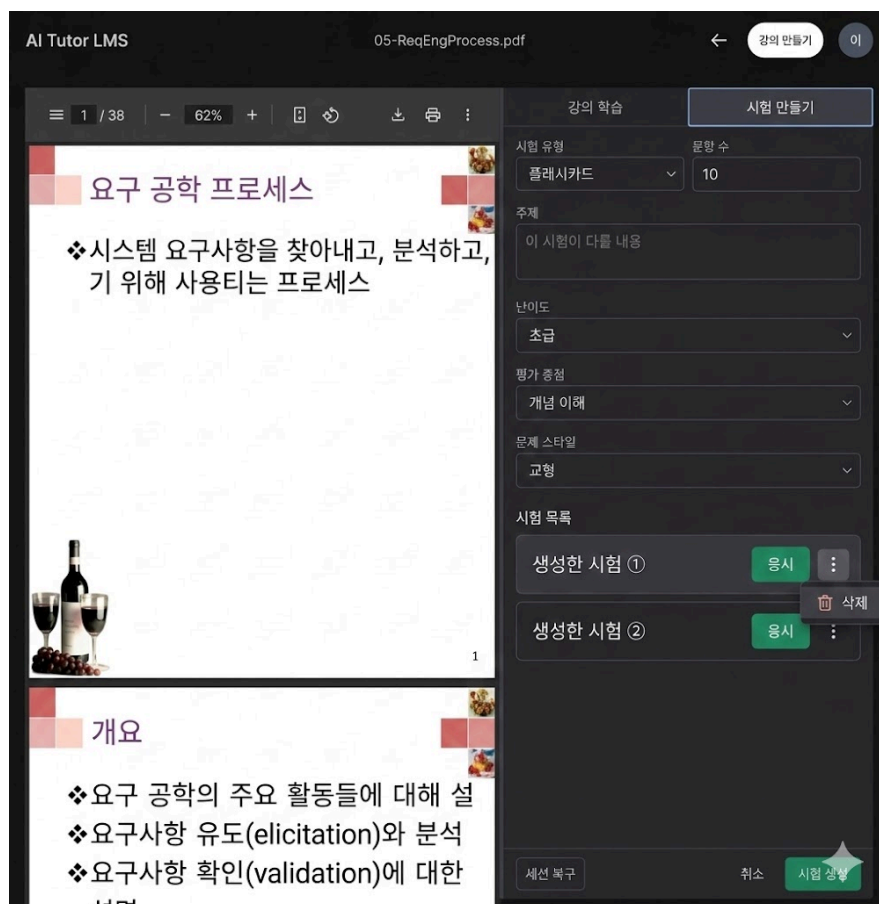
- 사용자가 질문을 입력하면, 답변은 두 단계로 나누어 출력된다.
 - 1단계: 사고 요약 표시
 - 2단계: 최종 답변 스트리밍 출력
- 답변 생성이 시작되면, 먼저 사고 요약 영역이 화면에 노출된다.
 - 이 영역은 반드시 토글 UI 내부에 표시된다.
 - 사용자는 이 단계에서 시스템이 어떤 방향으로 답변을 구성하고 있는지 짧게 확인할 수 있다.
- 사고 요약 출력이 완료되면, 토글은 자동으로 닫힌다.
 - 사용자가 수동으로 닫을 필요는 없다.
 - 이후 화면의 주 출력 영역으로 전환된다.
- 토글이 닫힌 직후, 최종 답변이 스트리밍 방식으로 출력된다.
 - 답변은 한 번에 전체 렌더링되지 않는다.
 - 문장 또는 텍스트 청크 단위로 순차적으로 표시된다.
 - 사용자는 일반적인 채팅형 응답처럼 자연스럽게 읽을 수 있어야 한다.
- 스트리밍이 진행되는 동안, 화면은 출력 길이에 맞춰 자동 스크롤되어야 한다.
 - 새 텍스트가 추가되면 현재 응답의 마지막 위치가 계속 보이도록 유지한다.
 - 사용자가 직접 스크롤을 내리지 않아도, 진행 중인 답변을 끊김 없이 읽을 수 있어야 한다.
- 전체 흐름은 아래와 같이 고정한다.
 - 질문 입력
 - 사고 요약 토글 노출
 - 사고 요약 출력
 - 사고 요약 완료
 - 토글 자동 닫힘
 - 최종 답변 스트리밍 시작
 - 스트리밍 길이에 맞춰 자동 스크롤 유지
 - 답변 종료

향후 추가해볼 만한 아이디어 (차후에 진행)

- 추후에는 사용자와 대화를 주고받는 과정에서 다음과 같은 정보를 점차 파악할 수 있다.

- 사용자의 이해 수준
- 어떤 설명 방식을 선호하는지
- 어떤 부분에서 자주 막히는지
- 이러한 정보를 별도의 컨텍스트 메모리에 저장한 뒤, 시스템 프롬프트에 반영하는 방식으로 개선할 수 있다.
- 이렇게 되면 다음과 같은 방향의 발전이 가능하다.
 - 사용자의 수준에 맞는 설명 제공
 - 사용자가 선호하는 표현 방식 반영
 - 사용자가 더 잘 이해하는 설명 스타일로 최적화
- 즉, 장기적으로는 맞춤형 설명 학습에 가까운 방향으로 발전시킬 수 있다.

시험 만들기 UX 개선 (완료 - 3.19)



- 시험 만들기 → 시험으로 수정
- 시험 메뉴 안에서 → 시험 만들기랑, 생성한 시험 모두 볼 수 있게 수정
 - 또한, 더보기 버튼을 통해 삭제할 수 있게 해야함
 - 다만, 이때 삭제 버튼은 플로팅 형태로 나타나도록

토론형의 비활성화 (완료 - 3.19)

토론형 시험 기능 운영 방향

- 현재 시험 생성 기능에는 토론형 유형이 포함되어 있다.

- 다만 토론형은 원래 사용자와 공개적으로 상호작용하며, 실제 대화를 주고받는 방식으로 진행되어야 한다.
- 즉, 단순히 결과만 제시하는 형태가 아니라, 사용자와의 대화 흐름 자체가 기능에 반영되어야 한다.

현재 문제점

- 지금 구현 상태에서는 이러한 대화형 상호작용 구조가 전혀 반영되어 있지 않다.
- 따라서 현재의 토론형 기능은 본래 의도한 방식대로 동작하지 않는 상태라고 볼 수 있다.

당분간의 운영 방안

- 현재는 우선적으로 신경 써야 할 다른 사항이 많다.
- 따라서 토론형 기능은 당분간 비활성화하는 방향이 적절하다.

시험 관련 사항

▼ OX 문제 (완료 - 3.24)

1. 현재 상태

- 현재는 문제에 대해 사용자가 답을 선택하는 방식이 아님
- “정답/해설 보기” 버튼을 누르면, 바로 정답과 해설을 확인할 수 있는 구조임

2. 현재 문제점

- 사용자가 직접 문제를 풀고 채점하는 흐름이 아님
- 정답을 스스로 선택해보는 과정 없이 바로 정답과 해설을 확인하게 됨
- 따라서 실제 문제 풀이 경험보다는, 단순 정답 확인 방식에 가까움
- 또한 채점하기 전에도 정답과 해설을 볼 수 있어, 문제 풀이 기능으로 보기 어려운 상태임

3. 수정 필요 사항

- 문제마다 사용자가 **O 또는 X**를 직접 선택할 수 있어야 함
- 사용자가 모든 문제에 대해 답을 선택한 뒤,
화면 맨 아래의 **채점하기** 버튼을 눌러 결과를 확인할 수 있어야 함
- 즉, 현재 구조는 단순 정답 확인 기능이고,
여기서 **실제 채점 기능이 추가되어야 함**
- 이때, **채점하기** 버튼을 누르기 전에는 정답 및 해설을 볼 수 없어야 함
- 따라서 현재 각 문제마다 있는 “정답/해설 보기” 버튼은 제거되어야 함

4. 원하는 동작 흐름

1. 사용자가 각 문제에서 **O 또는 X**를 선택함
2. 모든 문제에 답을 선택하면, 맨 아래의 **채점하기** 버튼이 활성화됨
3. 사용자가 **채점하기** 버튼을 누름
4. 채점 결과가 표시됨
5. 각 문제별로 정답 여부와 해설이 함께 표시됨

5. 채점하기 버튼 활성화 조건

- 채점하기 버튼은 항상 활성화되어 있으면 안 됨
- 사용자가 모든 문제에 대해 **O 또는 X** 중 하나를 선택했을 때만 활성화되어야 함
- 하나라도 선택하지 않은 문제가 있으면, 채점하기 버튼은 비활성화 상태여야 함

6. 채점 전 정답 및 해설 노출 제한

- 사용자가 **채점하기** 버튼을 누르기 전에는 정답과 해설을 확인할 수 없어야 함

- 즉, 문제를 푸는 단계에서는 정답 여부나 해설이 노출되면 안 됨
- 따라서 현재처럼 각 문제마다 따로 있는 **정답/해설 보기 버튼은 없어야 함**
- 정답 여부와 해설은 **채점이 완료된 이후에만** 보여야 함

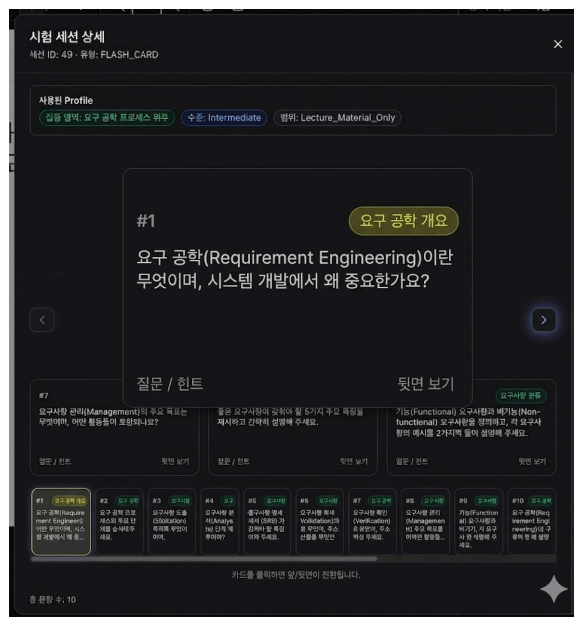
7. 채점 후 표시되어야 하는 내용

- 채점을 완료하면, 각 문제 바로 아래에 다음 내용이 표시되어야 함
 - 해당 문제의 **정답 여부**
 - 해당 문제의 **해설**
- 이 정보는 **문제마다 각각 따로 표시되어야 함**
- 즉, 전체 해설을 한 번에 묶어서 보여주는 것이 아니라, **각 문제 아래에 개별적으로 보여주는 방식이어야 함**

8. 점수 표시 방식

- 채점 후에는 사용자가 **몇 문제 중 몇 문제를 맞췄는지** 확인할 수 있어야 함
- 예를 들어 **10문제 중 8문제 정답**
이런 형태로 결과를 표시하면 됨
- 점수 또는 정답 개수 표시는 화면 맨 아래에 있는 **채점하기 버튼 바로 옆 또는 아래에 표시해야 함**

▼ FLASH_CARD (완료 - 3.24)



1. 현재 문제

- 현재 FLASH_CARD는 한 화면에 여러 내용이 보이는 방식이라 가독성이 떨어짐
- 카드 내용이 한눈에 잘 들어오지 않아서 보기 불편함
- 특히 뒷면 내용은 지금 방식으로는 더 보기 힘든 상태임

2. 수정 방향

- FLASH_CARD는 한 화면에 카드 1개씩만 보이도록 구성하는 방향이 좋아보임
- 사용자가 좌우 슬라이드 또는 상하 슬라이드 방식으로 다음 카드로 넘길 수 있게 하면 됨
- 즉, 여러 카드를 한 번에 보여주는 방식보다, **카드 1개에 집중해서 보는 구조로 바뀌어야 함**

3. 카드 앞면/뒷면 표시 방식

- 문제와 답변 모두 한 화면에서 더 잘 읽히도록 개선이 필요함
- 특히 뒷면은 내용이 길어질 수 있으므로, 현재보다 훨씬 읽기 쉽게 보여줘야 함

4. 뒷면 텍스트 표시 방식

- 뒷면 내용은 에이전트 응답 형태에 따라 다르게 처리하면 됨

4-1. 마크다운 형태로 응답이 오는 경우

- 에이전트의 답변이 마크다운 형식으로 들어온다면,
마크다운 문법을 그대로 적용해서 렌더링하면 됨

4-2. 일반 문장 형태로 오는 경우

- 에이전트의 답변이 지금까지 한 줄짜리 일반 문장 형태로 들어오는 경우에는,
최소한 마침표(.)가 나올 때마다 자동 줄바꿈이 적용되도록 해야 함
- 그래야 긴 문장이 한 줄로 붙어서 보이지 않고,
사용자가 내용을 훨씬 쉽게 읽을 수 있음

5. 글자 크기

- 한 화면에 카드 1개씩만 보이도록 바꾸는 만큼,
글자 크기도 지금보다 키우는 방향이 좋음
- 카드 내용을 빠르게 읽고 이해할 수 있도록,
문제와 답변 모두 가독성을 우선해서 보여줘야 함

▼ 5지선다

1. UI 방향

- 전체적인 UI 구성은 **지금 그대로 유지하면 됨**

2. 해설 표시 방식 수정

- 현재는 시험 채점 후, 화면 맨 아래에서 내가 선택한 답에 따른 해설을 한꺼번에 보여주고 있음
- 이 방식이 아니라, **각 문제 바로 아래에 정답 여부와 해설이 각각 표시되도록 해야 함**
- 즉, 해설은 문제별로 바로 확인할 수 있어야 함

3. 정답/해설 보기 버튼 제거

- 현재 각 문제마다 있는 **“정답/해설 보기” 버튼은 삭제해야 함**
- 정답과 해설은 문제를 푸는 도중에는 볼 수 없어야 함
- 오직 사용자가 **모든 문제를 푼 뒤, 채점하기 버튼을 눌렀을 때만** 정답과 해설을 볼 수 있어야 함

4. 채점하기 버튼 활성화 조건

- 현재는 사용자가 문제를 다 풀지 않았어도 채점하기를 누르면 바로 정답 여부와 해설이 표시되고 있음
- 이 방식은 수정해야 함
- **모든 문제에 답을 선택했을 때만 채점하기 버튼이 활성화되도록 해야 함**
- 하나라도 답을 선택하지 않은 문제가 있으면, 채점하기 버튼은 **비활성화 상태여야 함**

5. 채점 후 표시 내용

- 채점이 완료되면 각 문제 바로 아래에 아래 내용이 표시되어야 함
 - 해당 문제의 **정답 여부**
 - 해당 문제의 **해설**
- 이 정보는 문제마다 따로 보여줘야 함

6. 채점 결과 표시 위치

- 전체 채점 결과는 화면 맨 아래에 표시하면 됨

- 예를 들어 **10문제 중 8문제 정답**
이런 형태로 보여주면 됨
- 위치는 **채점하기 버튼 옆 또는 아래**면 됨

▼ 단답형/서술형

1. 정답 관련 정보 노출 방식

- 기존과 마찬가지로, 각 문제에 있는 “모범 답안/기준 보기” 버튼은 삭제해야 함
- 정답과 관련된 정보는 채점하기 버튼을 누른 이후에만 확인 가능해야 함
- 즉, 사용자가 문제를 푸는 도중에는 모범 답안, 채점 기준, 정답 관련 내용을 미리 볼 수 없어야 함

2. 핵심 키워드 / 출제 의도 노출 방식

- 핵심 키워드와 출제 의도는 처음부터 바로 보이면 안 됨
- 이 정보는 “키워드와 출제 의도 보기”
형태의 토크 UI 안에 넣어야 함
- 사용자가 직접 토크를 열었을 때만 내용을 확인할 수 있어야 함

3. 채점하기 버튼 활성화 조건

- 기존과 마찬가지로, 모든 문제에 대해 나의 답안이 작성되었을 때만 채점하기 버튼이 활성화되어야 함
- 하나라도 답안이 비어 있으면 채점하기 버튼은 비활성화 상태여야 함

4. 현재 채점 문제

- 현재는 채점 에이전트가 없어진 것으로 보임
- 또한 현재 도입된 키워드 기반 채점 방식은 한계가 커 보임
- 따라서 기존 방식 대신, Gemini AI 기반 채점 방식으로 새롭게 교체해야 함

5. 새로운 채점 방식

- 사용자가 채점하기 버튼을 누르면 Gemini AI가 각 문제별로 채점을 수행해야 함
- 이때 문제별로 아래 정보를 JSON 형태로 입력하면 됨
 - 문제 번호
 - 문제 내용
 - 핵심 키워드
 - 출제 의도
 - 사용자 답변
- 추가로, 현재 강의 자료 PDF도 함께 전달해야 함
 - PDF 전달은 pathlib 기반 방식으로 넘기면 됨

6. Gemini AI 역할

- Gemini는 문제별로 아래 역할을 수행해야 함
 - 문제 내용 파악
 - 핵심 키워드 파악
 - 출제 의도 파악
 - 사용자 답변 채점
 - 점수 부여
 - 채점 이유 작성
 - 피드백 작성

- 특히 점수가 깎인 경우에는,
왜 감점되었는지도 함께 알려줘야 함

7. Gemini 출력 형식

- Gemini의 출력은 반드시 JSON 응답으로 고정해야 함
- 문제별로 아래 정보가 포함되어야 함
 - 문제 번호
 - 점수
 - 채점 이유
 - 피드백
 - 감점 여부 또는 감점 사유

8. 점수 규칙

- 각 문제 점수는 0~1 사이의 실수값으로 가져야 함
- 최솟값은 0, 최댓값은 1임
- 점수 단위는 0.1, 0.2, 0.3, ...
형태로 처리하면 됨
- 최종 점수는 각 문제 점수 합 / 문제 수
형태로 계산하면 됨

9. 출력 방식 관련

- 강의 에이전트와 다르게,
사고 요약 과정 출력은 필요 없음
- 스트리밍 출력도 필요 없음
- 즉, 채점 결과만 최종적으로 정리해서 반환하면 됨

10. 채점 결과 표시 위치

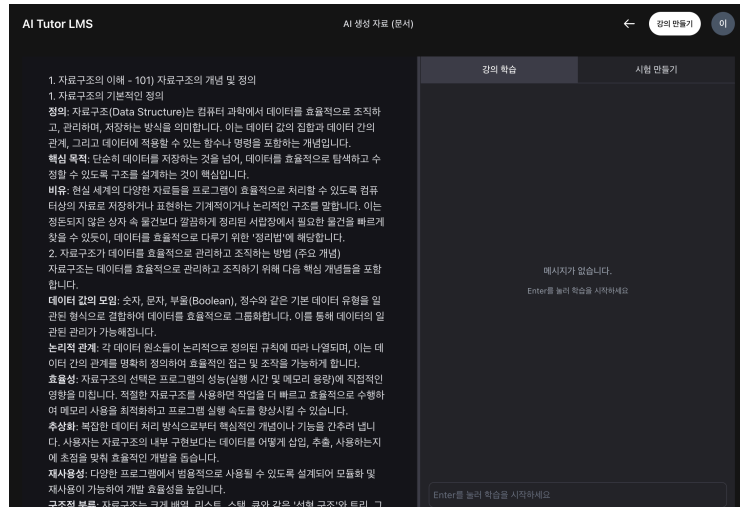
- 채점이 완료되면, 최종 점수는 채점하기 버튼 옆 또는 아래에 표시하면 됨
- 표시 형식은 예를 들어 아래처럼 하면 됨
 - 7.8 / 10
 - 총점 7.8 / 10

11. 문제별 결과 표시 방식

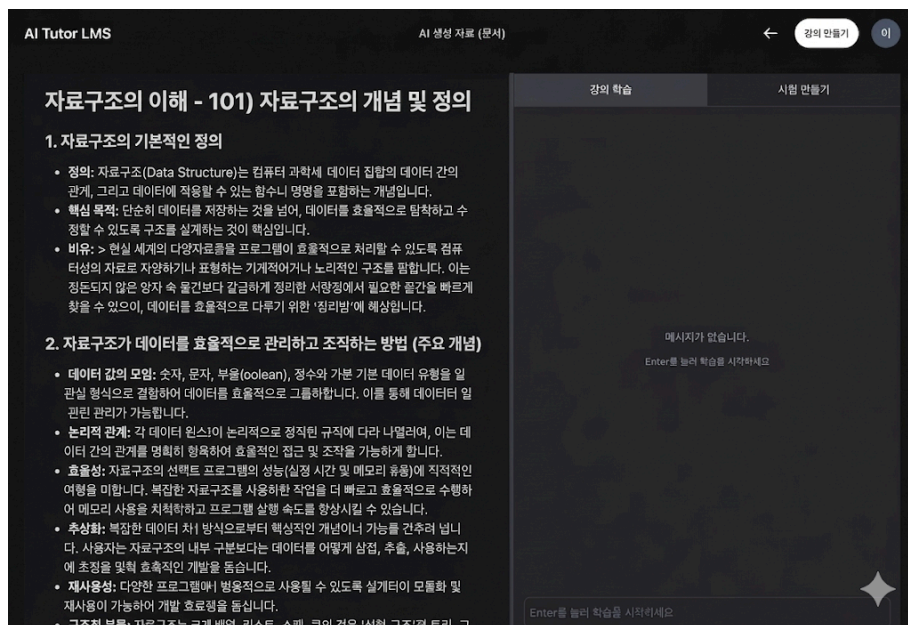
- 각 문제별 채점 결과는 해당 문제 바로 아래에 표시해야 함
- 즉, 문제마다 아래 내용이 바로 붙어서 보여야 함
 - 해당 문제 점수
 - 채점 이유
 - 피드백
 - 감점 사유(있다면)
- 지금처럼 화면 맨 아래에 모든 문제의 피드백과 결과를 한꺼번에 모아서 보여주는 방식은 제거함

마크다운 출력 관련

- 지금 AI 생성 자료에서 마크다운 출력이 안이루어진 것으로 확인

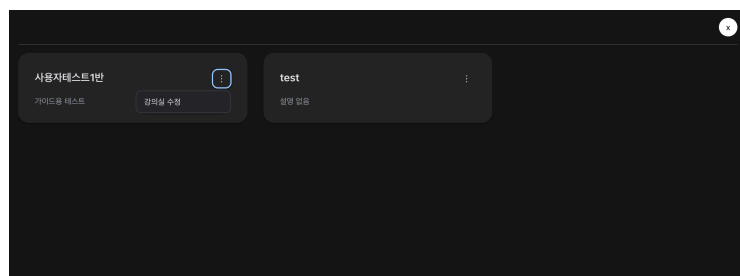


- 해당 메뉴를 마크다운 출력이 되도록 해줘야함

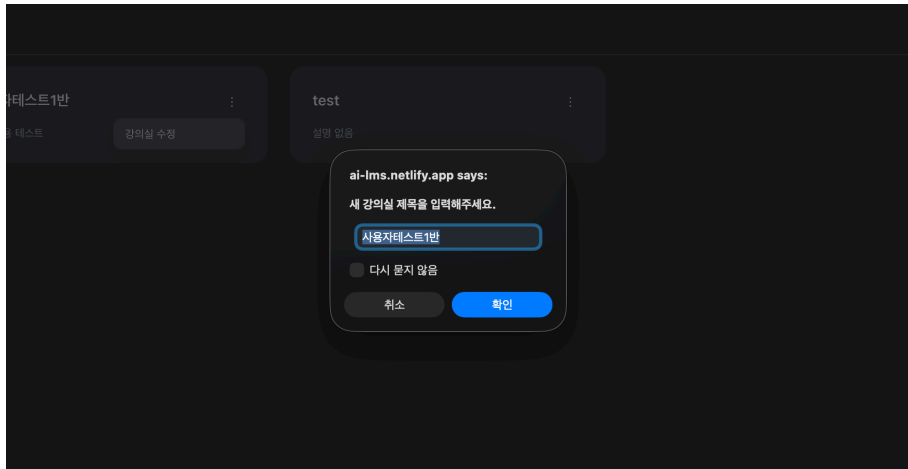


강의실 삭제 관련 (완료 - 3.19)

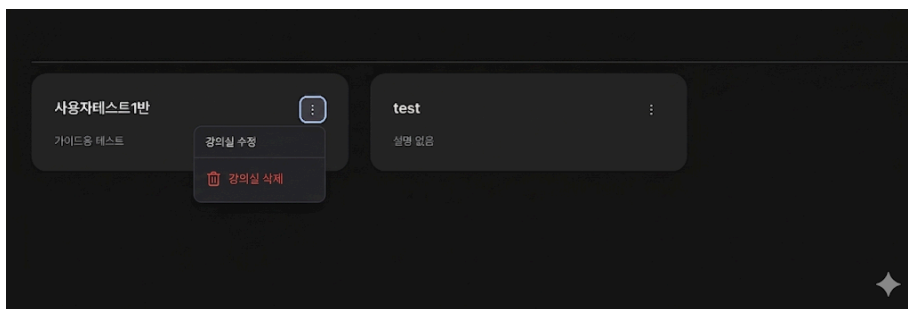
- 강의실 삭제가 안됨
- 수정 화면을 눌렀을때



- 이름 바꾸기 밖에 없음



- : 버튼을 눌렀을때 강의실 삭제 버튼이 나오도록 추가



시험 관련 개선 사항

- 현재 시험을 생성하면, 다른 PDF 리소스인데도 시험 메뉴에서 같은 시험이 그대로 보임
 - 예: 05-ReqEngProcess.pdf 에서 생성한 시험이 → AI 생성 자료(문서)에도 그대로 보이는 현상
- 시험 생성 및 관리는 해당 PDF 리소스 기준으로 분리되어야 함

1) 개선 방향

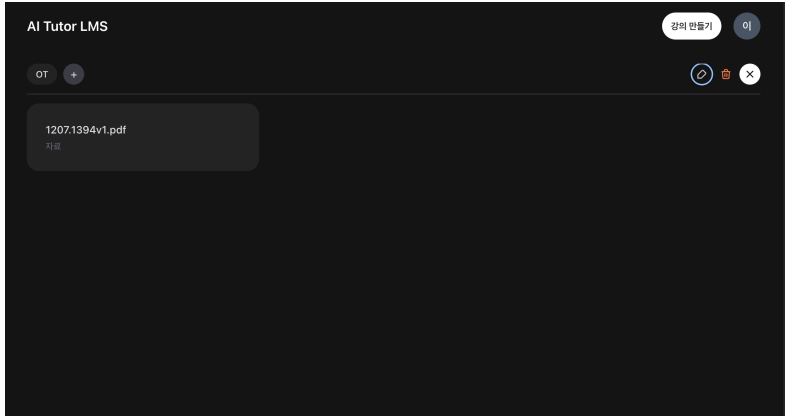
- PDF 리소스별로 시험을 따로 관리할 수 있게 수정
 - 시험 응시
 - 시험 삭제
 - 시험 추가
 - 시험 생성

2) 기대 동작

- 05-ReqEngProcess.pdf 에서 생성한 시험은 해당 PDF 강의실에서만 보여야 함
- 다른 PDF강의실에서는 그 PDF강의실에 연결된 시험만 보여야 함
- 즉, 시험은 전역으로 공유되는 게 아니라 PDF 리소스 단위로 관리되어야 함

강의 자료 수정 기능 관련 수정 요청

1) 현재 동작



- 강의실 내부 화면에서 수정 버튼을 누른 뒤,
- 수정하려는 강의 자료를 선택하면
- 해당 강의자료가 수정되는 것이 아니라, 해당 강의자료의 강의실로 이동됨

2) 문제점

- 사용자는 강의 자료를 수정하려는 의도로 선택한 것인데, 실제로는 상세 화면으로 이동하게 되어 혼란이 발생함
- 수정 기능과 이동 기능의 구분이 직관적이지 않음

3) 수정 필요 사항

- 수정 상태에서 강의 자료를 선택했을 때, 바로 해당 강의실로 이동하지 않도록 해야 함
- 강의실 이름을 수정할 수 있는 메뉴가 필요

4) 원하는 동작 흐름

- 현재: 수정 버튼 클릭 → 강의 자료 선택 → 해당 강의실로 이동함
- 수정안: 수정 버튼 클릭 → 강의 자료 선택 → 강의실 이름 변경 팝업 → 강의실 이름 변경

시험 이름 설정 관련 정리

AI Tutor LMS

1207.1394v1.pdf

< 1 / 8 > | 🔍 120% 🔍 📄 ⬇

Neur-optimal Nonmyopic Value of Information in Graphical Models

Andreas Krause
Carnegie Mellon University

Carsten Guestrin
Carnegie Mellon University

Abstract

A fundamental issue in real-world systems, such as sensor networks, is the selection of observations which most effectively reduce uncertainty. More specifically, we address the long-standing problem of automatically selecting the most informative subset of variables in a graphical model. We present the first efficient analysis and algorithm for computing the value of information (VI) for any $\epsilon > 0$ with high confidence. The algorithm leverages the theory of potential value functions, in combination with a polynomial-time approximation scheme. We furthermore prove that no polynomial-time algorithm can provide a constant factor approximation better than $1 - \epsilon$ unless $P = NP$.

Finally, we provide extensive evidence of the effectiveness of our method across complex real-world datasets.

1 Introduction

In decision making, where one can select between several information but expensive observations, it is often a central issue to decide which variables to observe in order to achieve a most effective increase in expected utility. We consider systems, such as sensor networks, where utility is usually associated with certainty about the measured quantity itself, and the task is to efficiently select the most informative subset of observations. Consider, for example, a temperature monitoring task, where various temperature sensors are distributed across a building as shown in Figure 1(a). Our goal in this example is to increase most certain about the temperature distribution, while minimizing energy expenditures, a critically constrained resource [15]. Unfortunately, as we show in [15], the problem of selecting the most informative subset of observations is NP^{VI} -complete, even when the underlying random variables are discrete and their prior probability distribution can be represented as a polytree graphical model (even though inference is efficient in these models). To address this complexity issue, it has been common practice (e.g., [20, 4, 1, 17]) to compute approximate solutions by using greedy algorithms to select the next observations or, equivalently, the set of observations with maximum joint entropy. Unfortunately, these greedy approaches are not associated with formal performance guarantees. Moreover, selecting observations with maximal joint entropy is an indirect measure of one value of information goal, the minimization of the remaining uncertainty after these observations are made. In some scenarios, for example, we are interested in a direct measure of value of information, e.g., minimizing the uncertainty about the measured quantities. We thus define our observation selection problem as that of selecting a subset of the possibly available V that maximizes information gain, $I(X; A) = H(X) - H(X|A)$, i.e., decrease in uncertainty about random variables.

Although there is a vast literature on approx. optimization for value of information (e.g., [20, 4, 1]), there has been little prior work on nonmyopic analysis. In [15], a method is proposed to compute the maximum expected utility for specific sets of observations. While their work considers more general objectives (because their information gain, they provide only single sample guarantees for the evaluation of a given sequence of observations), and use a heuristic without guarantees to select task objectives. In [12], we present efficient optimal algorithms for selecting subsets and sequences of observations for general objective functions, but these algorithms are restricted to chain graphical models.

In this paper, by leveraging the theory of submodular functions [16], we present the first efficient randomized algorithms providing a constant factor $(1 - \epsilon) - (1 - \epsilon)$ approximation guarantee for any $\epsilon > 0$ with high confidence, for any graphical model where inference can be performed efficiently. Our algorithm addresses both the joint entropy criterion, and, under weak assumptions, the information gain criterion. We furthermore prove that no polynomial-time algorithm can provide an approximation with a constant factor better than $1 - \epsilon$ unless $P = NP$. In addition to providing near-optimal information value selections, our formal guarantees can be used to automatically derive reliable performance guarantees for any other algorithm optimizing information gain.

Note that, in [15], we prove that computing the conditional entropy useful to an observer is NP -complete, even when the underlying distribution is represented as a polytree graphical model. We address this complexity problem in this paper by considering a different function which is computable in polynomial time. Our algorithm is a greedy algorithm with polynomial sample complexity. The algorithm of [16] requires $O(n^2)$ for maximizing submodular functions [16] can only deal with settings where every element has the same cost. In more general problems, different elements have different costs. Building on recent constant-

ai-lms.netlify.app - AI Tutor LMS

시험 유형
플래시카드

문항 수
10

주제
이 시험이 다룰 내용

난이도
초급

평가 중점
개념 이해

문제 스타일
균형

시험목록
FLASH_CARD · 10문항
FLASH_CARD · 10문항

세션 복구

시험 생성

1) 현재 상태

- 현재 시험 생성 시, 시험 이름은 자동으로 정해짐

피드백

13

- 예를 들어 flash card 형식으로 10문항을 생성하면 **FLASH_CARD · 10문항** 이런 형태로 이름이 생성됨
- 현재는 이 자동 생성 이름만 사용할 수 있음
- 시험 생성 과정에서 사용자가 **별도의 시험 이름을 직접 입력하는 기능은 없음**

2) 수정 필요 사항

- 시험 생성 메뉴에서 사용자가 **시험 이름을 직접 입력할 수 있도록 해야 함**
- 다만, 이름 입력은 **필수가 아니라 선택 사항**이어야 함
- 즉, 사용자가 이름을 직접 정할 수도 있고, 입력하지 않고 기존 방식으로 생성할 수도 있어야 함

3) 이름을 입력한 경우

- 사용자가 시험 생성 시 이름을 직접 입력하면, 입력한 이름을 시험 이름으로 사용하면 됨
- 다만, 사용자가 입력한 이름이 기존에 이미 존재하는 시험 이름과 동일하다면, **이미 존재하는 시험 이름이라는 알림이 출력되어야 함**
- 이 경우 사용자가 다른 이름으로 수정할 수 있도록 해야 함

4) 이름을 입력하지 않은 경우

- 사용자가 이름을 따로 입력하지 않으면, 기존 방식 그대로 자동 이름을 사용하면 됨
- 즉, **시험 형식 · 문항수** 형식으로 이름이 생성되면 됨

▼ 예시

- FLASH_CARD · 10문항
- OX_QUIZ · 20문항

5) 자동 이름 중복 처리 규칙

- 사용자가 이름을 입력하지 않았고, 자동 생성된 이름이 기존에 이미 존재하는 경우에는 뒤에 **(2), (3)** 형태를 붙여서 구분해야 함

▼ 예시

- FLASH_CARD · 10문항
- FLASH_CARD · 10문항 (2)
- FLASH_CARD · 10문항 (3)

6) 중복 처리 시 주의사항

- **(2), (3)** 같은 표시는 **시험 형식과 문항 수가 모두 동일한 경우에만** 붙여야 함
- 시험 형식 또는 문항 수가 다르면, 같은 이름으로 보지 않아야 함
- 따라서 조건이 일치하지 않으면 **(2), (3)** 등을 붙이지 않아야 함

▼ 예시

- 기존에 **FLASH_CARD · 10문항** 이 이미 있음
 - 다시 **FLASH_CARD · 10문항** 생성함
 - 이 경우 이름이 완전히 같으므로 **FLASH_CARD · 10문항 (2)** 로 생성하면 됨
- 기존에 **FLASH_CARD · 10문항** 이 이미 있음
 - 새로 **FLASH_CARD · 20문항** 생성함
 - 문항 수가 다르므로 같은 이름으로 보지 않음
 - 따라서 **(2)** 를 붙이지 않음
- 기존에 **FLASH_CARD · 10문항** 이 이미 있음
 - 새로 **OX_QUIZ · 10문항** 생성함
 - 시험 형식이 다르므로 같은 이름으로 보지 않음

- 따라서 (2) 를 붙이지 않음

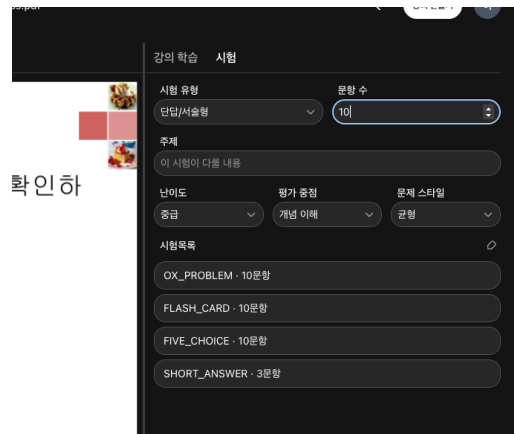
7) 이름 입력 필드 가이드 문구

- 시험 이름 입력 칸에는 사용자가 이름을 쉽게 정할 수 있도록 가이드 문구(placeholder) 를 함께 보여줘야 함

▼ 예시

- 개념 정리를 위한 플래시카드
- 중간고사 대비 암기용 문제
- 빠른 복습용 테스트
- 이런 형태의 예시 문구를 넣어서, 사용자가 시험 이름을 정할 때 어렵지 않게 작성할 수 있도록 해야 함

시험 생성에서 문항수 버그



1) 현재 상태

- 문항수 위/아래 버튼은 1단위로 정상 동작함
- 기본 문항수도 현재처럼 10개로 유지하면 됨

2) 현재 문제

- 사용자가 문항수 입력 칸을 마우스로 클릭한 뒤 키보드로 직접 입력하려고 하면 버그가 발생함
- 숫자를 입력할 때 값이 갑자기 20으로 튀는 문제가 있음
- 또한 일의 자리 입력이 정상적으로 되지 않음
- 예를 들어 기존 값이 10일 때,
 - 0 을 지우면 1 만 남음
 - 그런데 그 상태에서 1 이 더 이상 지워지지 않음
- 이 때문에 사용자가 원하는 숫자를 자유롭게 직접 입력할 수 없는 상태임

3) 수정 방향

- 기존 위/아래 버튼 방식은 그대로 유지해야 함
- 기본값도 현재처럼 10개로 유지하면 됨
- 다만, 사용자가 입력 칸을 클릭해서 키보드로 직접 숫자를 입력하려고 할 때는 값을 자유롭게 수정할 수 있어야 함
- 즉, 입력 중에는 지금처럼 값이 강제로 바뀌거나, 특정 숫자가 지워지지 않는 문제가 없어야 함

4) 입력 방식

- 사용자가 키보드로 직접 입력하는 동안에는 숫자를 자유롭게 수정할 수 있어야 함
- 입력 도중에는 값을 강제로 보정하지 말아야 함

- 입력값 검사는 사용자가 입력을 마친 뒤에만 처리하면 됨

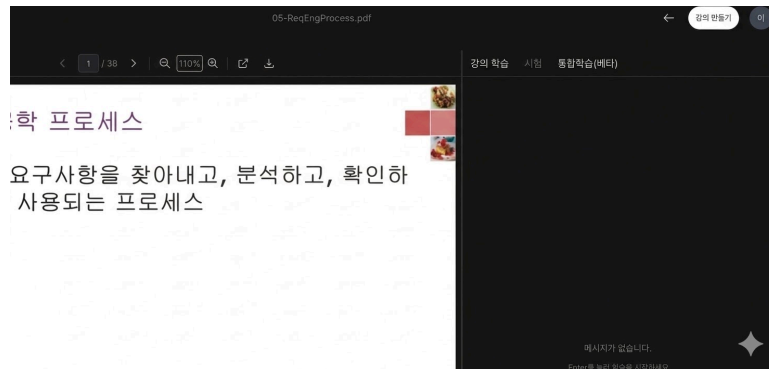
5) 팝업 처리 시점

- 팝업은 키보드 입력이 끝난 뒤에만 띄워야 함
- 아래와 같은 경우에 입력 완료로 판단하면 됨
 - 사용자가 Enter를 눌렀을 때
 - 사용자가 다른 버튼을 클릭했을 때
 - 사용자가 입력칸 바깥 다른 영역을 클릭했을 때

6) 예외 처리

- 입력이 끝난 뒤 값이 비정상적이면 그때 팝업을 띄우면 됨
- 예를 들어
 - 0 또는 음수 입력 시 → 비정상 입력 안내 팝업 출력함
 - 허용 범위를 초과한 값 입력 시 → 문항 수가 너무 많다는 팝업 출력함

통합 에이전트 접근 관련



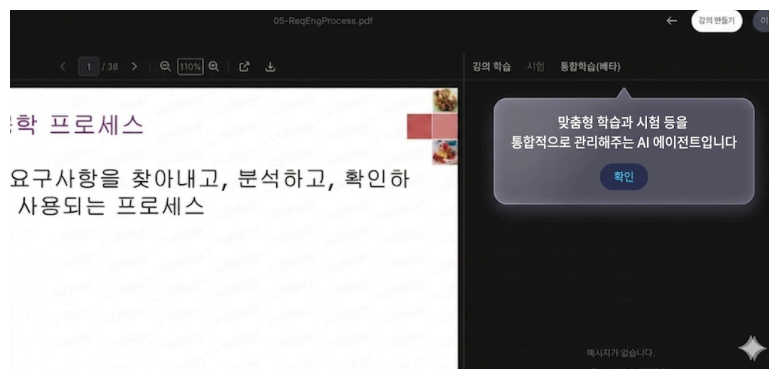
1) 메뉴 추가

- 현재 메뉴 구성에서 “시험” 바로 옆에 “통합학습(베타)” 메뉴를 추가하면 됨

2) 최초 방문 안내

- 사용자가 통합학습(베타) 메뉴를 처음 방문했을 때,
플로팅 형태의 안내 메시지를 보여주면 됨 + 확인 버튼이 있어야 함 → 확인버튼 클릭시 해당 플로팅 말풍선은 사라진다

3) 안내 문구



- 최초 안내 메시지는 아래와 같은 내용을 넣으면 됨

시험 생성에서 다른 PDF 자료로 만들어지는 버그

1) 현재 문제

- 지금은 어떤 강의실에서, 어떤 강의 자료로 시험을 만들든
시험 내용이 무조건 05-ReqEngProcess.pdf
이 파일 기준으로 생성되고 있음

2) 문제 상황

- 예를 들어
 - 05-ReqEngProcess.pdf 가 있음
 - 다른 강의 자료로 1207.1394v1.pdf 가 있음
- 이 상태에서 사용자가 1207.1394v1.pdf 안으로 들어가서 시험을 생성했는데도
실제 생성된 시험 내용은 05-ReqEngProcess.pdf 기준으로 만들어지고 있음

3) 영향 범위

- 이 문제는 같은 강의실 안의 다른 PDF끼리도 발생함
- 또한 다른 강의실에 있는 PDF에서도 동일하게 발생함
- 즉, 현재는 사용자가 어느 PDF에서 시험을 만들든,
실제 참조 대상이 특정 PDF 하나로 잘못 고정되어 있는 상태임

4) 정상 동작

- 시험은 사용자가 현재 들어가 있는 PDF 기준으로만 생성되어야 함
- 즉,
 - 1207.1394v1.pdf 에서 시험을 만들면 → 1207.1394v1.pdf 내용으로 시험 생성되어야 함
 - 05-ReqEngProcess.pdf 에서 시험을 만들면 → 05-ReqEngProcess.pdf 내용으로 시험 생성되어야 함

5) 수정 필요 사항

- 시험 생성 시, 현재 사용자가 선택한 해당 강의 자료 PDF를 정확히 참조하도록 수정해야 함
- 시험 생성 대상 PDF가 다른 파일로 잘못 고정되지 않도록 해야 함
- 각 강의실 / 각 PDF별로 시험은 무조건 현재 보고 있는 그 PDF 기준으로만 생성되어야 함